

Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (declared model gpt-5-mini + deterministic_netops_validator_v5). The preregistration allowed independent per-arm generation (up to 400 model calls) or shared_initial_candidate semantics (one LLM call per task); the executed run used shared_initial_candidate and thus used model_calls_used = 200. Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0, n00 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided p = 1.0; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced under shared semantics and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that a deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish a complete archived preregistered run, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats.

I. INTRODUCTION

Large language models (LLMs) are increasingly proposed to translate operator intent into device configuration changes for NetOps and AIOps workflows. While automating intent→configuration translation can improve velocity and reduce manual labor, incorrect or unsafe configuration proposals may cause outages, policy violations, or security exposures. A common mitigation is tool-augmentation: pair an LLM generator with deterministic validators, sandboxed simulators, or constrained executors in a generate–validate–repair pipeline so that proposed changes are checked before execution. We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether adding a deterministic verifier causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM on synthetic NetOps tasks. The primary estimand was the paired SAFE-rate difference (guarded minus baseline) tested with an exact McNemar two-sided test and a paired

bootstrap percentile CI (10000 resamples, seed = 1729). The preregistration permitted two generation semantics (independent per-arm or shared_initial_candidate); the executed run used shared_initial_candidate to conserve hosted model budget. Because the sampled candidates were valid in every confirmatory episode, the paired causal contrast was mechanically untestable; the manuscript therefore focuses on the completed run, a precise diagnosis of non-informativeness grounded in run evidence, and concrete, empirically actionable recommendations for informative repeats.

II. RELATED WORK

Prior surveys and prototypes document rapid uptake of LLMs for NetOps tasks (intent translation, configuration synthesis, diagnosis and limited remediation) and stress both potential utility and safety concerns [1–4]. Empirical and conceptual work on tool-augmentation shows that augmenting LLMs with deterministic validators, simulators, or constrained executors can reduce explicit constraint violations on curated tasks when the external verifier faithfully encodes domain constraints; reported gains, however, depend strongly on verifier fidelity and evaluation semantics (sampling rules, whether repairs are applied, and generation budgeting) [5–7]. Literature on guardrails, schema-based tool interfaces, and secure agent orchestration emphasizes practical trade-offs: stricter deterministic checks can lower acceptance of unsafe proposals but may increase false refusals or operator burden if the validator is incomplete or overly conservative; several papers advocate ROC-style diagnostics and adversarial stress-testing of guardrails to reveal these trade-offs [2,3]. Methodological work for agentic NetOps recommends workflow-centred, replayable, sandboxed experiments with explicit initial and expected final states, deterministic task generators, and archived validator traces so runs are reproducible and auditors can replay failures; these desiderata motivated our reproducible, preregistered design and informed the diagnostic emphasis of this report [1,5].

III. METHODOLOGY

Preregistration and estimand. The study was preregistered (cand-2026-01-prereg-v1). Primary estimand:

paired_success_rate_difference_guarded_minus_baseline computed on binary SAFE indicators produced by a deterministic sandbox validator. Confirmatory test: exact two-sided McNemar with a paired bootstrap percentile CI (10000 resamples, seed = 1729). The preregistration specified a paired binary design with $N=200$ tasks and a power plan targeting detection of ≈ 7 percentage-point reductions in unsafe proposals assuming a baseline unsafe rate $\approx 10\text{--}15\%$.

Task generation and scope. A deterministic generator produced 200 synthetic intent $\rightarrow$ configuration tasks constrained to single-AS multi-device topologies and limited configuration fields (admin, mtu, vlan). Generator seeds were deterministic and the task family encoded explicit initial_state, expected_final_state, and required_sequence workflow policies. Tasks included a mix of benign and adversarial prompt transforms per the preregistered adversarial template set intended to raise difficulty.

Compared pipelines and scoring. Baseline: accept the LLM candidate and score post hoc with deterministic_netops_validator_v5. Guarded: validate the same candidate in a deterministic sandbox using deterministic_netops_validator_v5; the validator emits structured validation_before/after traces, violation lists, and a deterministic valid flag (1 = SAFE, 0 = UNSAFE). For confirmatory episodes no automatic repairs were applied, so the guarded outcome reflects pure validation rather than repair-induced change.

Generation semantics and budgeting. The preregistration allowed independent per-arm generation (one model call per arm per task) or shared_initial_candidate (one shared model call per task scored by both arms). The executed run used shared_initial_candidate to conserve hosted model budget; consequently model_calls_used = 200 while independent per-arm generation would have used up to 400 calls. Under shared semantics and without repairs, both arms necessarily scored the identical candidate for each task.

Analysis. Deterministic validator outputs were converted to binary SAFE labels. Paired analysis computed the 2×2 contingency counts ($n_{11}, n_{10}, n_{01}, n_{00}$), exact McNemar two-sided p-value, and paired bootstrap percentile CI (10000 resamples, seed = 1729). Missingness and retry rules from the preregistration were applied: one automatic retry per failed model call and deterministic exclusion/replacement rules for contaminated or nondeterministic tasks. Deterministic reconciliation checks and provider accounting were performed to confirm a single reproducible execution without technical failures.

IV. RESULTS

Execution accounting. The confirmatory execution completed the planned paired episodes (200 pairs; 400 episodes) with no failed episodes. Because shared_initial_candidate semantics were used, model_calls_used = 200 (one LLM call per task) rather than the preregistered independent per-arm maximum of 400.

Primary confirmatory outcomes. The deterministic validator labeled every sampled candidate SAFE: baseline SAFE =

200/200 and guarded SAFE = 200/200. The resulting paired contingency counts are $n_{11} = 200$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$. The paired SAFE-rate difference (guarded - baseline) = 0.0. The preregistered exact McNemar two-sided test yields $p = 1.0$. The paired bootstrap percentile 95% CI is degenerate at $[0.0, 0.0]$ (resamples = 10000; bootstrap_seed = 1729). These numeric outputs are the direct confirmatory analysis results.

Representative diagnostic traces. Validator traces from multiple tasks show identical shared_initial_candidate content and deterministic validation_before/after records with violation_count = 0 and valid = true for each sampled candidate. Repair traces are null for all confirmatory episodes, confirming that no automatic repairs were invoked and that the guarded arm performed only validation.

Missingness and reserves. All pairs were complete (complete_pairs = 200) with zero failed or unscorable episodes. A preregistered exploratory reserve of up to 50 tasks intended for adversarial calibration and ROC estimation was not consumed prior to confirmatory execution; consequently no empirical ROC or refusal-rate estimates appear in the confirmatory analysis.

Statistical interpretation. The ceiling outcome ($n_{11} = 200$ with no discordant pairs) renders McNemar’s test and the paired contrast non-informative for the intended causal estimand: the test requires discordant pairs (n_{10} and n_{01}) to evaluate within-pair differences. The degenerate contingency (no discordant pairs) produces $p = 1.0$ and a degenerate CI at zero, but this statistical degeneracy does not demonstrate verifier effectiveness—rather, it reflects that, under the executed shared generation semantics and for the sampled model outputs, there was no arm-level variation to attribute to the presence or absence of the deterministic validator.

V. DISCUSSION

Root cause synthesis. The degenerate contingency arises from three joint facts recorded in the execution: (1) generation semantics: the run executed shared_initial_candidate, producing a single LLM candidate per task that both arms scored; (2) validator outcomes: deterministic_netops_validator_v5 labeled every sampled candidate as valid (baseline SAFE = 200/200); and (3) absence of repairs: the guarded arm did not alter any candidate. Together these make the paired causal contrast mechanically untestable because the arms had no opportunity to disagree.

Design implications. A paired estimand that attributes differences to adding a deterministic validator requires per-arm variation that can produce discordant outcomes. Under shared generation semantics, discordance can only arise from repair-driven candidate changes in the guarded arm or non-deterministic validator behavior; neither occurred here. When budgets force a choice, the estimand should drive budgeting: independent per-arm generation is necessary to test the causal effect of adding a validator to generation. For this study size ($N=200$), independent per-arm generation requires roughly 200 additional model calls (400 total) compared with the executed shared run (200 calls).

Artifact-grounded recommendations for informative repeats. All recommendations follow directly from the preregistered design and the observed execution facts: (i) use independent per-arm generation for confirmatory paired contrasts unless the guarded arm will deterministically repair candidates in a way that reliably produces candidate changes; (ii) consume a preregistered exploratory reserve (we permitted up to 50 tasks) to calibrate adversarial transforms until the baseline unsafe rate matches power assumptions, and freeze those transforms before the confirmatory run; (iii) if repairs are enabled in the guarded pipeline, archive and analyze repair traces to quantify UNSAFE→SAFE conversions and check for repair-introduced violations; and (iv) implement a deterministic pilot check step on a small holdout that inspects contingency counts prior to full confirmatory execution—if the pilot yields a ceiling/floor contingency, abort and adapt along preregistered contingency-preserving paths (e.g., enable independent per-arm draws or strengthen adversarial transforms).

Alternative estimands. When budgets or policy preclude independent per-arm draws, choose an estimand testable under shared semantics: estimate the LLM baseline absolute unsafe-proposal rate with appropriate CIs (single-arm proportion); treat the guarded pipeline as an accept/reject gate and estimate refusal and post-repair SAFE rates (requires repairs); or use the exploratory reserve to estimate ROC trade-offs using injected UNSAFE ground truth. These alternatives shift resource needs but preserve inferential validity and are explicitly enumerated in the preregistration.

Threats to generalization. The deterministic validator here encodes `full_intent_constraint_validation` for the synthetic task family and ran deterministically in sandboxed states; its completeness relative to production failure modes is limited. Tasks restricted fields to `admin/mtu/vlan` and single-AS topologies; therefore zero unsafe detections in this sample do not imply production safety in heterogeneous multi-vendor, multi-AS environments. Future studies should validate validators against broader operational rule sets and include human adjudication for ambiguous cases, with such extensions preregistered.

Operational implications. Strict guardrails can reduce deployment risk but may increase false refusals and operator workload. Our confirmatory run produced no refusals and therefore does not measure that trade-off; this was the purpose of the preregistered exploratory ROC reserve. Deployment teams must weigh the cost of additional model calls (for independent draws or calibration) against the operational risk of executing unverified changes; the resource arithmetic linking `N`, per-arm draws, and `model_call` budgets should be made explicit in preregistrations and governance decisions.

Reproducibility note. The execution and analysis were archived as a deterministic, replayable run. Investigators seeking to repeat or extend this experiment should consult the archived preregistration and execution artifacts to preserve comparability and to avoid repeating a non-informative shared-generation execution for a paired causal estimand.

VI. LIMITATIONS

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by `deterministic_netops_task_generator_v5` and limited to single-AS, multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.
- Generation semantics: the executed run used `shared_initial_candidate` (`independent_condition_generation = false`), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from `gpt-5-mini` on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (`gpt-5-mini`) was evaluated; no multi-model generality is claimed.
- Verifier completeness: `deterministic_netops_validator_v5` ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.
- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. CONCLUSION

We executed a fully archived preregistered paired experiment comparing `gpt-5-mini` baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed `shared_initial_candidate` semantics, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0; exact McNemar $p = 1.0$; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The paper’s contribution is therefore methodological and reproducibility-centred: (1) a deterministic preregistered run that completed without technical failures and that is archived for replay; (2) an artifact-grounded diagnosis of why the confirmatory contrast was non-informative; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration via an exploratory reserve, and pilot contingency checks) for informative repeats. Future confirmatory work should align generation semantics and budgets with the estimand, and preregister exploratory calibration steps to ensure the targeted baseline unsafe rate is realized before committing to a paired test.

DISCLOSURE STATEMENT

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: `gpt-5-mini`. Orchestration/framework: `hosted_netops_gvr_v1` adapter and deterministic experiment harness. Exact initial master prompt: archived at `provenance/master_prompt.txt` with immutable SHA-256 =

1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba.

Topic constraints and autonomy boundary: the human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock; after the lock the autonomous researcher performed literature review, preregistration, experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution and recorded the primary estimand (paired SAFE-rate difference), paired design N=200, allowed generation semantics, scoring rules, and stopping rules. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock.

REFERENCES

- [1] <https://arxiv.org/abs/2605.12729>
- [2] <https://doi.org/10.2139/ssrn.7132138>
- [3] <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>
- [4] <https://doi.org/10.36227/techrxiv.177006109.97957916/v1>
- [5] <https://doi.org/10.1109/ojcoms.2026.3680457>
- [6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhang Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709
- [7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026